

Flask Dashboard, Predict CLI and Integration

A Surrogate ML Model for a Ti-6Al-4V Cementless Hip Implant

Ali Edip Mangtay
Software Engineering · 210911044

May 2026
ENS003 — Digital Twins for Health Sciences · Instructor: Prof. Dr. Şenol Pişkin

In this talk

- How I exposed the surrogate to an end user
- Architecture choices (Flask + Jinja + Plotly + KaTeX)
- The LaTeX-paper aesthetic and why we went that way
- Sidebar UX: three continuous sliders + three preset buttons
- Two production-flavoured bugs (HTTP 500, cache)

My responsibility (software team)

The web app, the predict CLI, templates/static, integration, `run.sh`.

Architecture

Server (Python):

- Flask + Jinja2 (HTML rendering)
- joblib model loading (10 models at init)
- Plotly figure JSON generation
- / (HTML) and /predict (JSON) endpoints

Client (browser):

- Plotly.js (CDN, chart rendering)
- KaTeX (CDN, equations)
- Vanilla JS slider events
- `fetch('/predict?...')` for AJAX

Design call: no bundler, no framework, two libraries via CDN. Reason: for an academic deliverable we want **minimum dependencies** — the demo should run on the jury's laptop in seconds.

The LaTeX-paper aesthetic

The dashboard was designed to look like an academic article:

- Centered title-block, authors, instructor, date
- An italic-grey *Abstract* section
- Numbered sections (1. Loading conditions, 2. Predicted quantities, ...)
- Inline and display equations rendered with KaTeX
- Booktabs-style tables (1.6pt top/bottom rules, thin mid rule)
- STIX Two Text serif family (academic PDF feel)

Why bother?

For the jury demo, we did not want a plain “ML form” — we wanted an **interactive extension of the project report**. Our UX lines have to match the Beamer/LaTeX aesthetic.

Sidebar UX

Three continuous sliders:

- mass — 40–110 kg
- K — 2.0–6.0 (body-weight multiplier)
- angle — 0° – 20° (force tilt)

Three quick-jump preset buttons:

- Walking $\Rightarrow K = 2.5$
- Stair climbing $\Rightarrow K = 3.5$
- Running / jumping $\Rightarrow K = 4.5$

Preset buttons only set the K slider; mass and angle are left alone. Activity is not a categorical input here — K is a continuous parameter, so a dropdown would impose a fake discreteness.

Three SF gauges:

- Neck
- STEM 1
- STEM 2

Status callout: driven by $\min(\text{neck}, \text{stem1}, \text{stem2})$ — labelled Safe / Caution / Critical.

The predict endpoint

```
@app.route("/predict")
def predict_endpoint():
    try:
        mass = float(request.args.get("mass", 70.0))
        K = float(request.args.get("k", 2.5))
        angle = float(request.args.get("angle", 0.0))
    except (TypeError, ValueError):
        return jsonify(error="invalid query parameters"), 400
    return jsonify(build_payload(mass, K, angle, models, train_df))
```

build_payload runs all 10 models and returns:

- input: $(m, K, \theta, F_x, F_z, |F|)$
- metrics: 3 SFs + 3 stresses + deformation, all $\pm\sigma$
- status: Safe / Caution / Critical
- near: closest FEA row (normalised (m, K, θ) distance)
- figures: 6 Plotly figure JSON objects

Bug story: “The angle slider does nothing”

Report: the dashboard **freezes** when `angle` moves 0→20.

Failure chain:

- 1 User picks `angle=5` (grid is `{0, 10, 20}`, off-grid)
- 2 In `comparison_chart`, the filter `np.isclose(angle, 5)` returns **zero rows**
- 3 Empty pandas Series gets handed to Plotly as an ndarray
- 4 Flask jsonify $\Rightarrow$ **500: ndarray not JSON serializable**
- 5 Frontend `if (!resp.ok) return;` $\Rightarrow$ **silent fail**
- 6 The user sees stale values $\Rightarrow$ “nothing changed”

Fix: `_nearest_value()` snaps (K, θ) to the closest grid point, the legend label adapts, and `.tolist()` casts guard the serializer.

Snap to nearest slice

```
def _nearest_value(value, options):  
    return float(options[int(np.argmin(np.abs(options - value)))]))  
  
def comparison_chart(rows, ..., K, angle, ...):  
    K_grid = np.sort(rows["K_factor"].unique())  
    angle_grid = np.sort(rows["force_angle_deg"].unique())  
    K_near = _nearest_value(K, K_grid)  
    angle_near = _nearest_value(angle, angle_grid)  
  
    same = rows[  
        np.isclose(rows["K_factor"], K_near)  
        & np.isclose(rows["force_angle_deg"], angle_near)  
    ].sort_values("patient_mass_kg")  
    # ... add Scatter trace with .tolist() casts
```

The legend label adapts automatically:

- Exact grid point: “FEA training (K=2.5, =0°)”
- Off-grid: “FEA training (nearest slice K=2.5, =0°)”

Dashboard screenshot

OPERATING POINT

Patient mass m 70 kg

Body-weight multiplier K 4.5×BW

Force tilt angle θ 20°

QUICK PRESETS

Walking
 $K = 2.5$

Stair
Climbing
 $K = 3.5$

Running /
Jumping
 $K = 4.5$

Material
Ti-6Al-4V (annealed)
 $\sigma_y = 827$ MPa
 $\sigma_u = 918$ MPa
 $E = 104.8$ GPa

A Surrogate Model for Stress and Safety Prediction in a Cementless Ti-6Al-4V Hip Implant

Ata Bulut · Recep Kamil Fırat · Ali Edip Mangtay
Department of Computer / Software Engineering, Istinye University
ENS003 — Digital Twins for Health Sciences · Instructor: Şenol Pişkin
May 2026

ABSTRACT

We present a Gaussian-process surrogate model trained on 270 ANSYS Static Structural simulations (162 unique grid points) of a Ti-6Al-4V cementless hip-implant stem. The training grid sweeps six patient masses (50–100 kg), nine body-weight multipliers $K \in \{2, 2.5, \dots, 6\}$, and three force-tilt angles $\theta \in \{0^\circ, 10^\circ, 20^\circ\}$. Given a patient mass, K and θ , the surrogate predicts three minimum safety factors (neck, STEM 1, STEM 2), the maximum von Mises and principal stresses, and the maximum total deformation in milliseconds. 5-fold cross-validation grouped on (m, K, θ) yields $R^2 \approx 0.999$ across all load-bearing targets.

Status — Caution. Marginal: the safety margin is low; experimental verification is recommended.

1. Loading conditions

The femoral force is parameterised by the body-weight multiplier K and the in-plane tilt angle θ :

$$F_x = m g K \cos \theta, \quad F_z = m g K \sin \theta, \quad \|\mathbf{F}\| = m g K.$$

For the operating point selected in the sidebar — $m = 70$ kg, $K = 4.5$, $\theta = 20^\circ$, $F = 3,090$ N — the resulting force is applied to the femoral head.

The surrogate consumes only three independent inputs: patient mass, K , and θ . The force components F_x and F_z shown above are derived analytically from those three — the model never sees activity labels. All ANSYS model constants (mesh, geometry, material, 18-face fixed support) are held fixed across runs and therefore not exposed as inputs.

2. Predicted quantities

Polish details

- **Cache disabled:** `SEND_FILE_MAX_AGE_DEFAULT=0` $\Rightarrow$ every CSS/JS edit shows up on a normal refresh (dev productivity).
- **Mobile sidebar:** `@media (max-width: 760px)` gives a slide-in panel; hamburger toggle handles open/close.
- **Posterior $\pm\sigma$ annotations:** GPR's standard deviation appears in italic grey next to every gauge and stress bar.
- **Activity preset buttons:** they only set the K slider — mass and angle stay where the user left them.
- **Training table scroll + sticky thead:** 270 rows, 460px height, headers stay pinned while scrolling.

CLI + integration

```
$ ./run.sh predict --mass 70 --k 2.5 --angle 0
{
  "safety_factor_neck_min": 1.553,
  "safety_factor_stem1_min": 1.662,
  "safety_factor_stem2_min": 5.711,
  "max_equivalent_vonmises_stress_Pa": 532000000.0,
  ...
}

$ ./run.sh predict --mass 80 --activity stair_climbing
# Snaps K to the preset, then predicts.
```

Both the dashboard and the CLI call the same `predict()` function.
`preprocess.add_features()` is the **single source of truth**.

Deployment paths (open decision)

Today: `flask run` dev server, Python deps ~ 234 MB.

Vercel/Netlify serverless: 250 MB unzipped limit — right at the edge, with 3–5 s cold starts.

Candidate routes:

- ➊ Hugging Face Spaces / Render / Railway (Flask as-is)
- ➋ Push Plotly client-side (saves ~ 40 MB)
- ➌ **Static precompute:** sample (m, K, θ) once, ship ~ 1 MB gzipped JSON, do trilinear interpolation in the browser $\Rightarrow$ GitHub Pages, zero backend.
- ➍ Port the GPR to JS (~ 100 lines), exact predictions, no server.

The team will pick a path after this talk.

Thank you.

Questions?

- `src/app.py`, `src/predict.py`, `src/templates/`, `src/static/`
- `run.sh` (CLI wrapper)
- `scripts/capture_screenshots.py` (dashboard renders)
- Sprint log: `docs/PROJECT_NOTES.md` Sprint 4, 7